ITDA Sizing

Team Mehg

March 2025

1 Known Parameters

1.1 Aerostat Body

1.1.1 Envelope

Material	LLDPE
GSM	100 g/cm^2
Shape	Oblate Spheroid
Major Dia.	6 m
Minor Dia.	3.75 m

Table 1: Envelope Specifications

The volume and surface area of the envelope shape are given by:

$$V_{oblate} = \frac{4}{3}\pi a^2 c \tag{1}$$

$$S_{\text{oblate}} = 4\pi \left[\frac{a^{2p} + 2a^p c^p}{3}\right]^{\frac{1}{p}}, p = 1.6075 \tag{2}$$

Where a is the semi-major axis and c is the semi-minor axis. From table 1, $a = 3$m, and $c = 1.875$m.

1.1.2 Sail

Material	LLDPE
GSM	52 g/cm^2
Shape	Isosceles triangle
Base	8 m
Height	3 m

Table 2: Sail Specifications

1.1.3 Ribbon

Material	Rip Stop Nylon
Linear Density	33 g/m

Table 3: Ribbon Specifications

Ribbon	Circumference
D1	1.8 m
D2	13.6 m
D3	17.3 m
D4	18.3 m
D5	17.3 m
D6	13.6 m
D7	1.8 m

Table 4: Horizontal Ribbons

Ribbon	Circumference	Number of Pieces
B1	4.3 m	8
H1	7 m	16

Table 5: Vertical Ribbons

1.2 LTA Gas

Gas	H_2
Purity (Beginning of Mission)	0.99
Leak Rate	0.25 l/m^2/day

Table 6: LTA Gas Specifications

1.3 Payload

Item	Quantity	Weight
RF Omni Antenna	1	4 kg
RF Cable	70 m long	0.133 kg/m
Confluence Lines	8, starting @ max. width to 2.5 m below	4 g/m
Handling Lines	$4 * 10$ m long	4 g/m
Safety Lines	2 from max. width to ground	4 g/m
Tether	1 (braided nylon)	5 g/m
Payload Bay	1	2.25

Table 7: Payload Specifications

1.4 Operating Conditions

Floating Altitude	60 m
Envelope Overpressure (above atm. pressure)	500 Pa (N/m^2)
Deployment Time	16 hours
Deployment Location	Ahmednagar: 19°07'48.7"N 74°44'49.4"E
Lat, Long	19.13019444, 74.74705556
Location Altitude	674.4 m
Max. Temp	32 °C
Min. Temp	16 °C
Relative Humidity	67%

Table 8: Operating Specifications

2 Tasks

2.1 Task 1

↪ **Mass of the Envelope system including Sail and ribbon-based Structure**

Using equation (2), $S_{oblate} = 86.221$ m^2, and GSM of envelope material is 100 g/m^2.

$$m_{env} = gsm * S_{oblate}$$
$$m_{env} = 8622.1g = 8.6221kg$$

Using table 2,

$$m_{sail} = gsm * S_{sail}$$
$$m_{sail} = 52\text{g/m}^2 * \frac{bh}{2}$$
$$m_{sail} = 52\text{g/m}^2 * 12\text{m}$$
$$m_{sail} = 624\text{g} = 0.624\text{kg}$$

Using tables 3, 4, 5,

$$m_{ribbon} = \lambda * l_{ribbon}$$
$$m_{ribbon} = 33\text{g/m} * [2(D_1 + D_2 + D_3) + D_4 + 8B_1 + 16H_1]$$
$$m_{ribbon} = 33\text{g/m} * [2(1.8 + 13.6 + 17.3) + 18.3 + 8 * 4.3 + 16 * 7]\text{m}$$
$$m_{ribbon} = 7593.3\text{g} = 7.5933\text{kg}$$

Mass of envelope system $= m_{env} + m_{sail} + m_{ribbon} = 14.7597$kg

2.2 Task 2

↪ **Mass of Confluence, Safety & Handling lines, Tether and RF cable**

I was confused about weather the floating altitude of 60m in table 8 is from the ground to the payload or from the ground to the bottom of the aerostat. According to the sources in the bibliography, the operating altitude is the height at which you want to operate your payload, and the following calculations have been performed with this assumption.

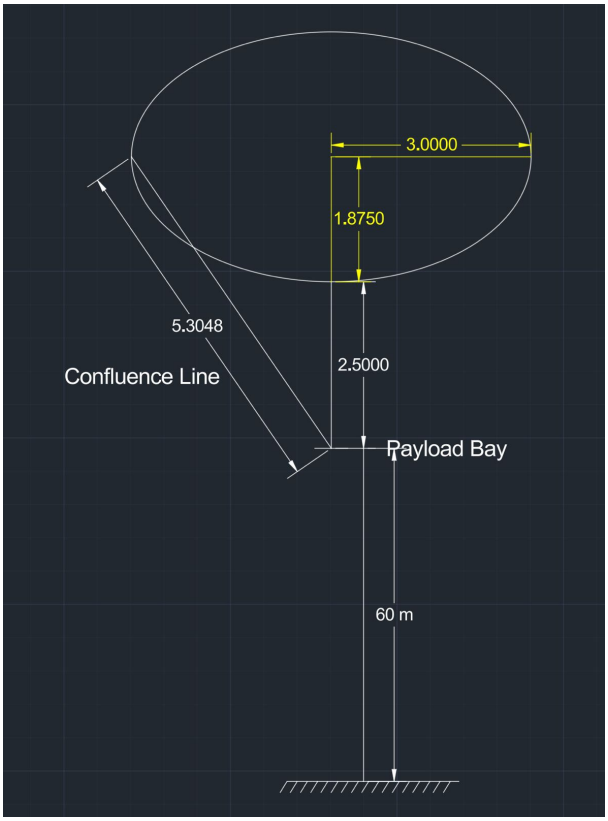


Figure 1: Length of Lines

Line	Quantity	Linear Density (g/m)	Length (m)	Weight (g)
Confluence	8	4.00	5.3048	169.75
Safety	2	4.00	64.375	515.00
Handling	4	4.00	10.000	160.00
Tether	1	5.00	60.000	300.00
RF Cable	1	133.00	70.000	9310.00
			Total Weight (kg)	**10.45**

Table 9: Mass Calculation of Lines

2.3 Task 3

↪ **Maximum and Minimum value of Net Static Lift during deployment period**

Considering isothermal conditions, the pressure at the height of the location can be found by,

$$P_A = P_o \exp\left(\frac{-gM(h-h_0)}{RT}\right) \tag{3}$$

Pressure at location altitude of 674.4 m at min. temp. 16°C = 93560.73 Pa
Pressure at location altitude of 674.4 m at max. temp. 32°C = 93952.65 Pa

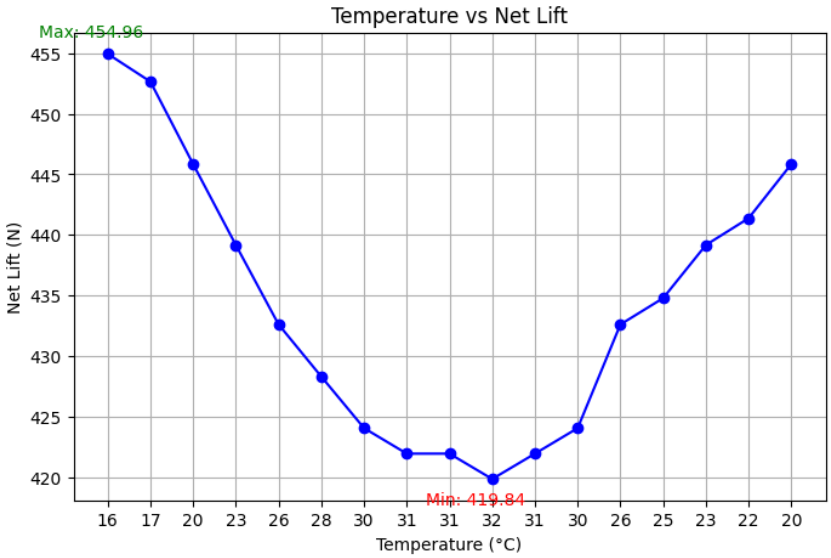
The rest of the calculations have taken condition 1 to be that with min. temp of 16°C and condition 2 to be with maximum temp of 32°C.

Using $\rho = \frac{MP}{RT}$, where $P = P_A + \Delta P_{\text{SP}}$ the density of the lifting gas and ambient air was found.

Parameter	Value	Note
ρ_{lg1}	0.078250735	w/SP
ρ_{lg2}	0.074456739	w/SP
ρ_{A1}	1.13344937	w/SP
ρ_{A2}	1.07849395	w/SP
ρ_{gas1}	0.088802721	Mixture Inside Env.
ρ_{gas2}	0.084497111	Mixture Inside Env.

Table 10: Density Parameters

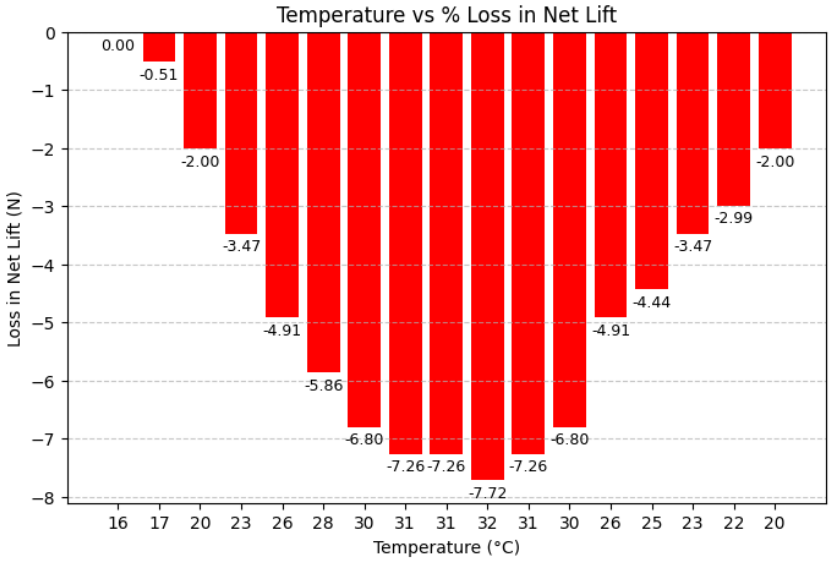
Weight of lifting gas was found simply by $m = \rho V$ and lifting gas weight and payload weights were added up.
Total volume of the envelope from equation 1 is 70.68583 m^3.



$\implies$ Maximum net static lift at the start of the day when temperature was minimum.

2.4 Task 4

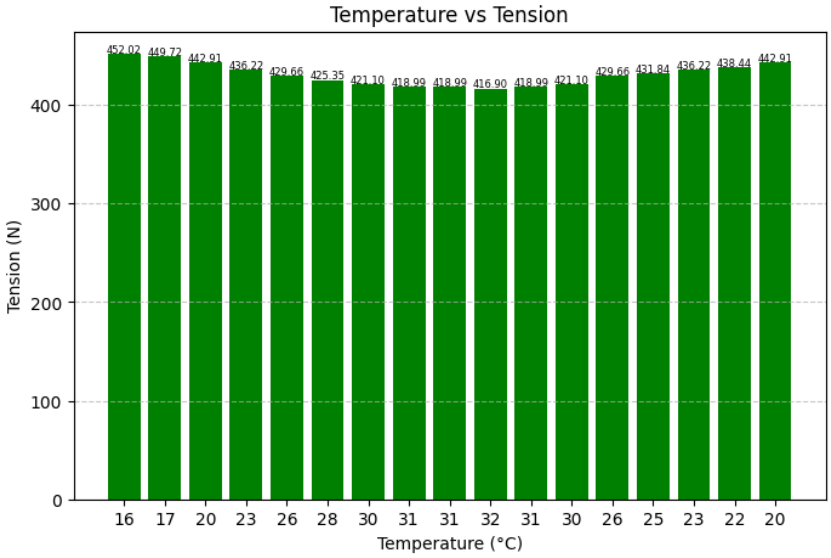
↪ % Loss in Net Static Lift over the deployment period



2.5 Task 5

↪ Minimum and Maximum Tension in the Tether and Factor of Safety

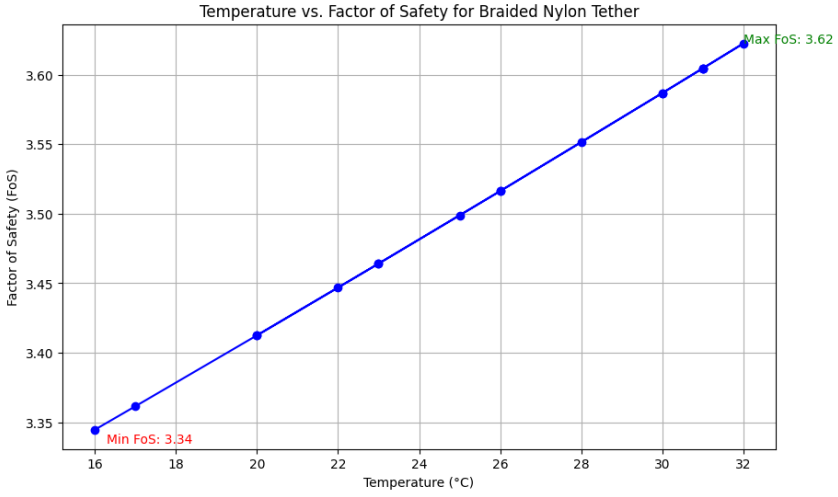
Tension in tether = net lift of aerostat - self weight of tether (0.3 kg)



Max Tension = 452.02 N @ 16°C
Min Tension = 416.90 N @ 32°C

Taking the tensile strength σ_{nylon} as 7.8×10^7 from here.
Assuming diameter of tether to be 5 mm. and performing the following calculations:

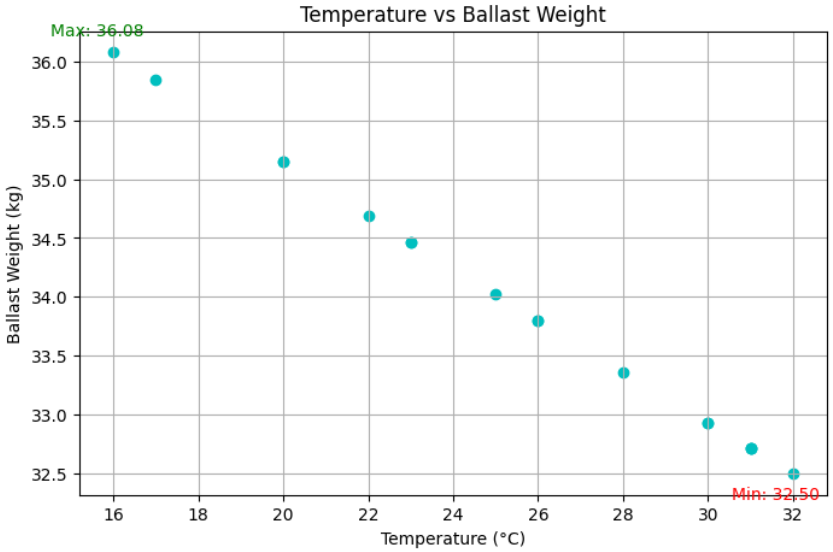
```
F_total = net_lift + 0.3*g  # Applied force (N)
A = np.pi * (d / 2) ** 2
F_ultimate = sigma_nylon * A
FoS = F_ultimate / F_total
```



2.6 Task 6

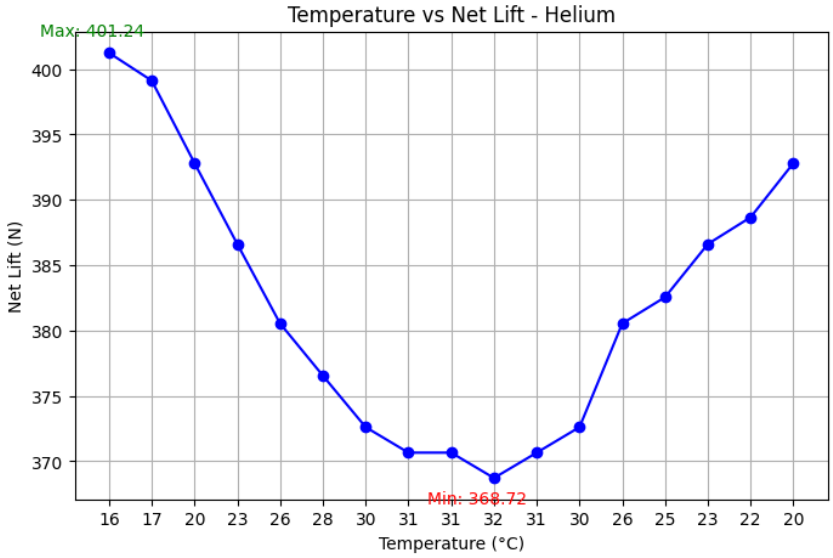
↪ Maximum and Minimum Ballast required to maintain tension of 10 kg in the Tether

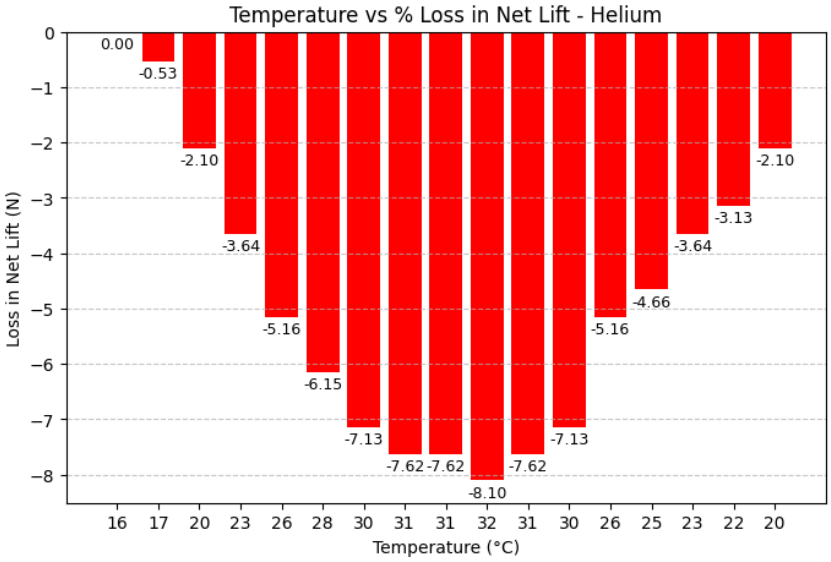
Tension in tether = $T \implies \frac{T}{g} - \text{ballast(kg)} = 10$

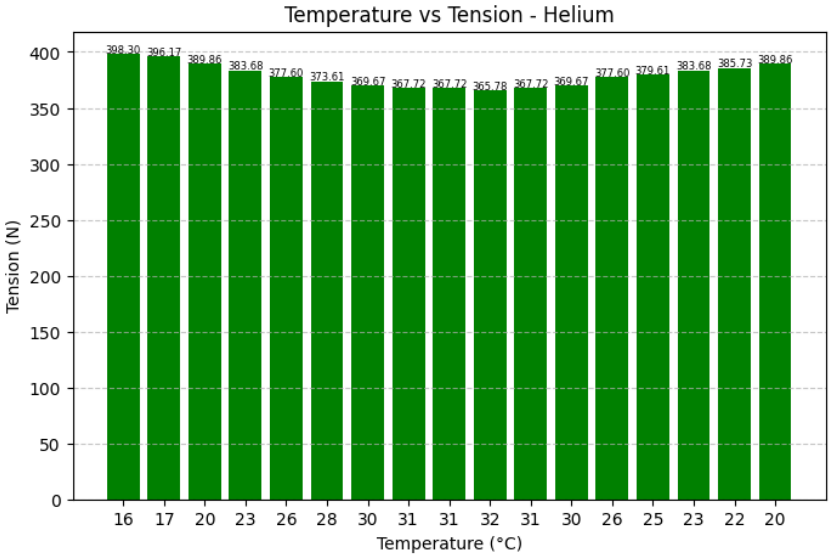


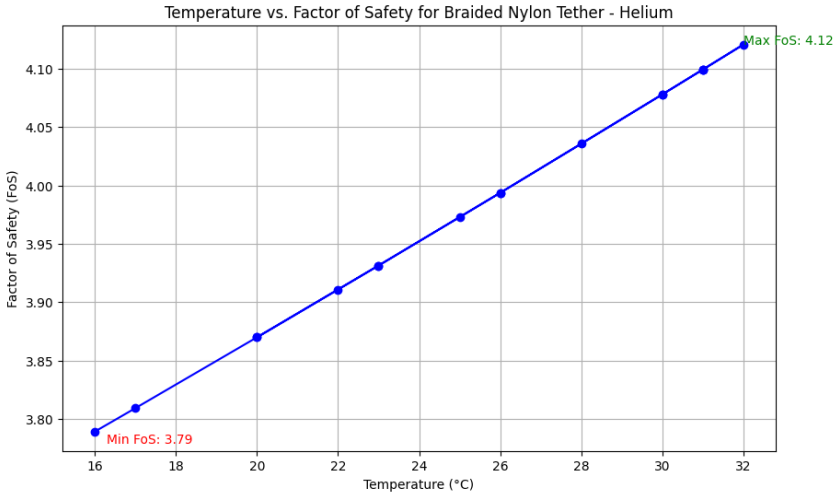
2.7 Task 7

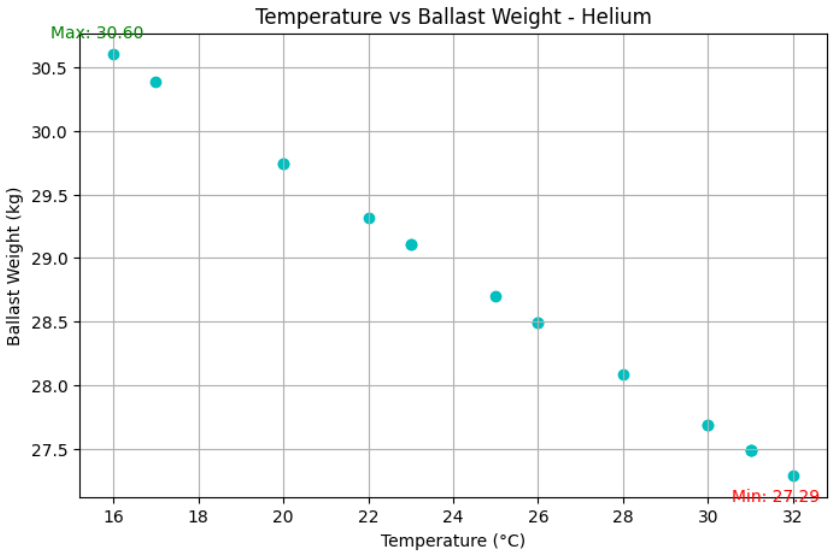
↪ **Repeat the above calculations from 3. onwards, if the lifting gas is changed to Hydrogen**




Temperature vs % Loss in Net Lift - Helium
Loss in Net Lift (N)
Temperature (°C)
0.00
-0.53
-2.10
-3.64
-5.16
-6.15
-7.13
-7.62
-7.62
-8.10
-7.62
-7.13
-5.16
-4.66
-3.64
-3.13
-2.10
16 17 20 23 26 28 30 31 31 32 31 30 26 25 23 22 20


Temperature vs Tension - Helium
Tension (N)
Temperature (°C)
398.30
396.17
389.86
383.68
377.60
373.61
369.67
367.72
367.72
365.78
367.72
369.67
377.60
379.61
383.68
385.73
389.86
16 17 20 23 26 28 30 31 31 32 31 30 26 25 23 22 20


Temperature vs. Factor of Safety for Braided Nylon Tether - Helium
Factor of Safety (FoS)
Temperature (°C)
Max FoS: 4.12
Min FoS: 3.79



References

[1] Kumar, A., Sati, S. C., & Ghosh, A. K. (2016). Design, Testing, and Realisation of a Medium Size Aerostat Envelope. *Defence Science Journal*, *66*(2), 93. https://doi.org/10.14429/dsj.66.9291

[2] Ram, C. V., & Pant, R. S., Multidisciplinary Shape Optimization of Aerostat Envelopes.

graphs

April 1, 2025

[132]:
```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
```

[152]:
```
P_0 = 101325
DeltaP_SP = 500
g = 9.81
h = 674.4
M_air = 0.02896968
R = 8.31432
payload = 10.4547536 + 14.7597 + 2.25 # structure + lines + payload bay
v_env = 70.68583 # m^3
M_hydrogen = 0.002  # kg/mol for hydrogen (H2)
Y = 0.99
```

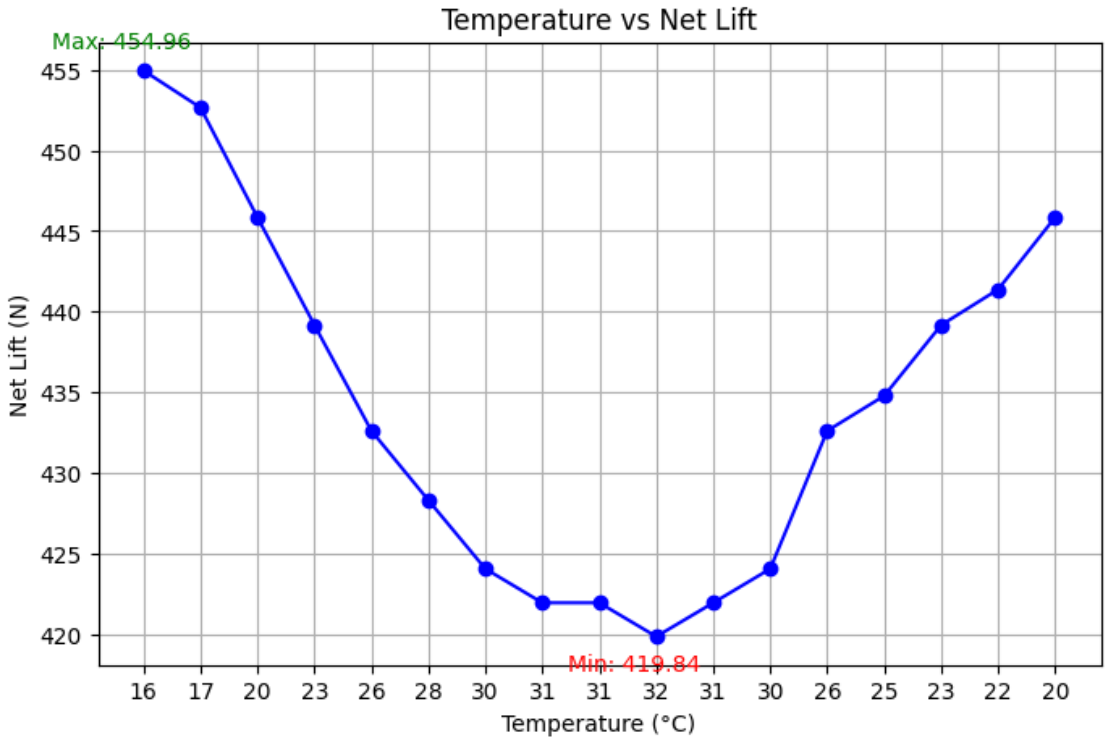
[153]:
```
temps = [16,17,20,23,26,28,30,31,31,32,31,30,26,25,23,22,20]
net_lifts = []

for t in temps:
    T = 273.15 + t
    P_A = P_0 * np.exp(-g * M_air * h / (R * T))

    P_effective = P_A + DeltaP_SP

    # Calculate the density of hydrogen (kg/m^3):
    rho_hydrogen = (M_hydrogen * P_effective) / (R * T)
    rho_air = (M_air * P_effective) / (R * T)

    rho_mixture = Y * rho_hydrogen + (1-Y) * rho_air

    # Calculate the mass of lifting gas:
    m_lg = v_env * rho_mixture

    gross_lift = v_env * rho_air * g

    total_weight = (payload + m_lg)
    net_lift = gross_lift - (total_weight * g)
    print(t, total_weight, gross_lift, net_lift)
```

```
    net_lifts.append(net_lift)
```

```
16 33.74154763360079 785.9655209415681 454.96093865594435
17 33.72162353719693 783.4707912584503 452.66166435854836
20 33.662600282521964 776.080390085558 445.8502813140175
23 33.60467626612472 768.8276263896516 439.1657522189681
26 33.54782118105196 761.7087053860483 432.6045795999286
28 33.51049717444032 757.0353036267129 428.29732634545337
30 33.47362685269511 752.4187085139699 424.04242908903086
31 33.45535925603998 750.1313919556156 421.9343176538634
31 33.45535925603998 750.1313919556156 421.9343176538634
32 33.437202024095065 747.8578943485937 419.8389424922211
31 33.45535925603998 750.1313919556156 421.9343176538634
30 33.47362685269511 752.4187085139699 424.04242908903086
26 33.54782118105196 761.7087053860483 432.6045795999286
25 33.56665593094175 764.0670361565765 434.7781414740379
23 33.60467626612472 768.8276263896516 439.1657522189681
22 33.62386402790805 771.2301583741989 441.38005226042094
20 33.662600282521964 776.080390085558 445.8502813140175
```

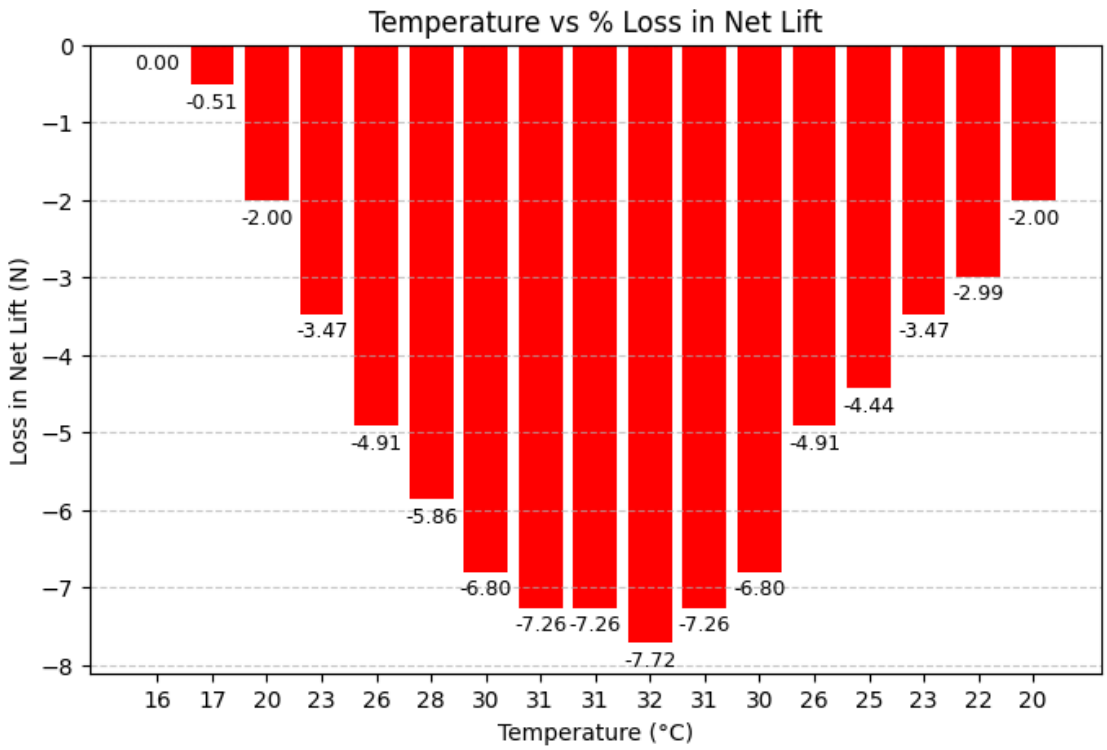
[154]:
```
indices = range(len(temps))

plt.figure(figsize=(8,5))
plt.plot(indices, net_lifts, marker='o', linestyle='-', color='b')

# Set tick labels to maintain order
plt.xticks(indices, temps)
plt.xlabel('Temperature (°C)')
plt.ylabel('Net Lift (N)')
plt.title('Temperature vs Net Lift')
plt.grid(True)

# Find max and min values
max_idx = net_lifts.index(max(net_lifts))
min_idx = net_lifts.index(min(net_lifts))

# Annotate max and min points
plt.annotate(f'Max: {net_lifts[max_idx]:.2f}',
             (max_idx, net_lifts[max_idx]),
             textcoords="offset points", xytext=(-10,10),
             ha='center', fontsize=10, color='green')

plt.annotate(f'Min: {net_lifts[min_idx]:.2f}',
             (min_idx, net_lifts[min_idx]),
             textcoords="offset points", xytext=(-10,-15),
             ha='center', fontsize=10, color='red')
```

```
plt.show()
```

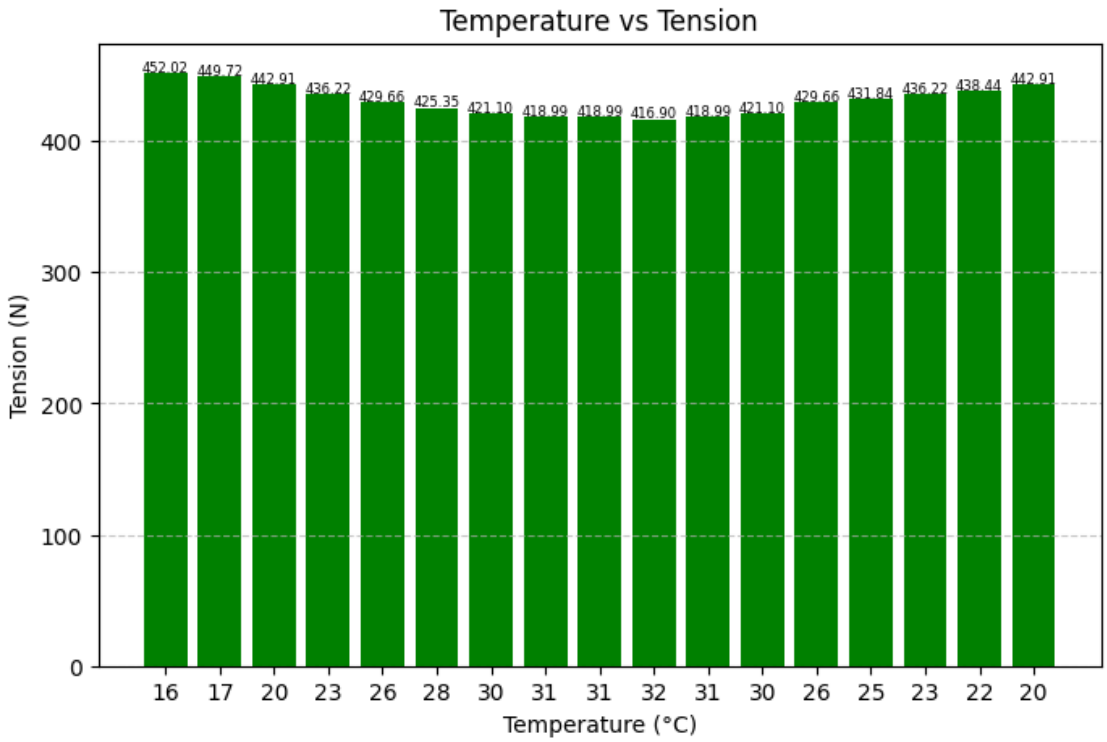


[155]:
```
max_lift = net_lifts[0]
loss_in_lift = []

for i, t in enumerate(temps):
    lost_lift = (net_lifts[i] - max_lift) / max_lift * 100
    loss_in_lift.append(lost_lift)

indices = range(len(temps))

plt.figure(figsize=(8,5))
plt.bar(indices, loss_in_lift, color='r')

# Set tick labels to maintain order
plt.xticks(indices, temps)
plt.xlabel('Temperature (°C)')
plt.ylabel('Loss in Net Lift (N)')
plt.title('Temperature vs % Loss in Net Lift')
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Label each bar with its value
```

```
for i, loss in enumerate(loss_in_lift):
    plt.text(i, loss - .3, f'{loss:.2f}', ha='center', fontsize=9)

plt.show()
```

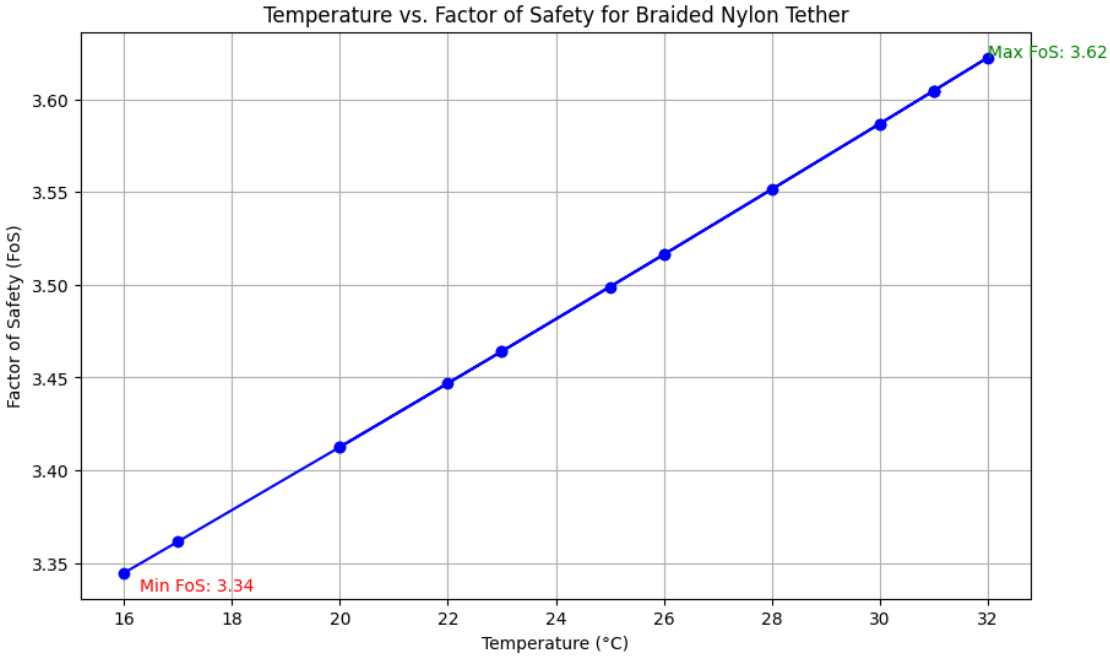


[156]:
```
tensions = [net_lift - 0.3*g for net_lift in net_lifts]

plt.figure(figsize=(8,5))
plt.bar(indices, tensions, color='g')

# Set tick labels to maintain order
plt.xticks(indices, temps)
plt.xlabel('Temperature (°C)')
plt.ylabel('Tension (N)')
plt.title('Temperature vs Tension')
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Label each bar with its value
for i, tension in enumerate(tensions):
    plt.text(i, tension + 1, f'{tension:.2f}', ha='center', fontsize=6)

plt.show()
```



[157]:
```
d = 5e-3  # Diameter in meters (5 mm)
sigma_nylon = 7.8e+7  # Tensile strength of braided nylon (N/m^2)

FoSs = []

for net_lift in net_lifts:
    F_total = net_lift + 0.3*g  # Applied force (N)
    A = np.pi * (d / 2) ** 2
    F_ultimate = sigma_nylon * A
    FoS = F_ultimate / F_total

    FoSs.append(FoS)


plt.figure(figsize=(10, 6))
plt.plot(temps, FoSs, marker='o', linestyle='-', color='b')
plt.xlabel('Temperature (°C)')
plt.ylabel('Factor of Safety (FoS)')
plt.title('Temperature vs. Factor of Safety for Braided Nylon Tether')
plt.grid(True)

# Annotate maximum and minimum FoS values
max_fos = max(FoSs)
```

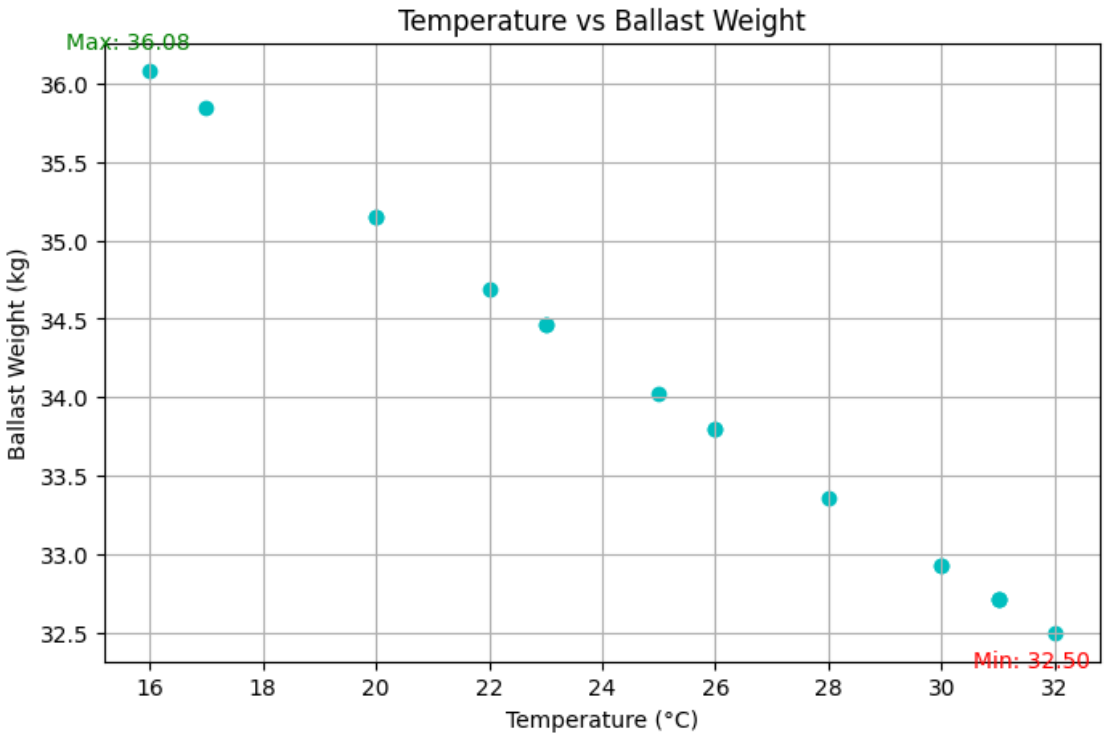
```
min_fos = min(FoSs)
max_temp = temps[FoSs.index(max_fos)]
min_temp = temps[FoSs.index(min_fos)]
plt.annotate(f'Max FoS: {max_fos:.2f}', xy=(max_temp, max_fos),␣
 ↪xytext=(max_temp, max_fos), fontsize=10, color='green')
plt.annotate(f'Min FoS: {min_fos:.2f}', xy=(min_temp, min_fos),␣
 ↪xytext=(min_temp+0.3, min_fos-0.01), fontsize=10, color='red')

plt.show()
```



[158]:
```
ballast_weights = []
for tension in tensions:
    ballast_weight = tension / g - 10
    ballast_weights.append(ballast_weight)
```

[159]:
```
# Create the scatter plot
plt.figure(figsize=(8,5))
plt.scatter(temps, ballast_weights, color='c', label='Ballast Weight')

# Set labels and title
plt.xlabel('Temperature (°C)')
plt.ylabel('Ballast Weight (kg)')
plt.title('Temperature vs Ballast Weight')
plt.grid(True)
```

```
# Find max and min values
max_idx = ballast_weights.index(max(ballast_weights))
min_idx = ballast_weights.index(min(ballast_weights))

# Dynamically adjust annotation offsets
def dynamic_xytext(idx, y_val, direction='up'):
    if direction == 'up':
        return (-10, 10)
    elif direction == 'down':
        return (-10, -15)
    # Example for other cases
    return (5, -5)

# Annotate max and min points dynamically
plt.annotate(f'Max: {ballast_weights[max_idx]:.2f}',
             (temps[max_idx], ballast_weights[max_idx]),
             textcoords="offset points",
             xytext=dynamic_xytext(max_idx, ballast_weights[max_idx], 'up'),
             ha='center', fontsize=10, color='green')

plt.annotate(f'Min: {ballast_weights[min_idx]:.2f}',
             (temps[min_idx], ballast_weights[min_idx]),
             textcoords="offset points",
             xytext=dynamic_xytext(min_idx, ballast_weights[min_idx], 'down'),
             ha='center', fontsize=10, color='red')

plt.show()
```


Temperature vs Ballast Weight
Max: 36.08
Min: 32.50
Ballast Weight (kg)
Temperature (°C)
36.0
35.5
35.0
34.5
34.0
33.5
33.0
32.5
16
18
20
22
24
26
28
30
32

[ ]: